

Protocolo de Camada de Aplicação

1. Objetivo

Este documento especifica um protocolo de camada de aplicação sobre TCP para validar números de CPF e CNPJ. O servidor é implementado em Python e o cliente em Java.

2. Visão geral do fluxo

O cliente abre uma conexão TCP, envia uma mensagem, aguarda uma resposta e encerra a conexão. O servidor aceita conexões simultâneas, cria uma thread para cada conexão autorizada e controla o limite máximo de conexões ativas.

3. Mensagem de requisição

O cliente envia um objeto JSON com os campos obrigatórios type, document_type e value:

Campo	Tipo	Descrição
type	texto	Deve ser validate.
document_type	texto	Deve ser CPF ou CNPJ.
value	texto	Documento com ou sem pontuação.

Exemplo de requisição CPF

```
{"type": "validate", "document_type": "CPF", "value": "529.982.247-25"}
```

Exemplo de requisição CNPJ

```
{"type": "validate", "document_type": "CNPJ", "value": "11.222.333/0001-81"}
```

4. Mensagem de resposta

Para uma requisição válida, o servidor responde com um objeto JSON contendo cpf_cnpj, valid e message:

Campo	Tipo	Descrição
cpf_cnpj	texto	Documento normalizado, somente com dígitos.
valid	booleano	true se válido; false se inválido.
message	texto	Mensagem do resultado da validação.

```
{"cpf_cnpj": "52998224725", "valid": true, "message": "CPF válido"}
```

```
{"cpf_cnpj": "52998224726", "valid": false, "message": "CPF inválido"}
```

5. Respostas de erro

Erros de transporte ou de formato são retornados como JSON com type, code e message:

Código	Significado
INVALID_JSON	A linha não pode ser interpretada como JSON.
INVALID_MESSAGE	O JSON recebido não é um objeto.
INVALID_VALIDATE	Campos ausentes ou document_type/value inválidos.
UNKNOWN_MESSAGE_TYPE	O campo type é diferente de validate.
MESSAGE_TOO_LARGE	A mensagem ultrapassa 4096 bytes.
SERVER_BUSY	O limite de conexões ativas foi atingido.

```
{"type": "error", "code": "INVALID_VALIDATE", "message": "dados de validação inválidos"}
```

6. Regras de validação

O servidor remove todos os caracteres não numéricos. CPF deve resultar em 11 dígitos e CNPJ em 14 dígitos. Sequências formadas por um único dígito são rejeitadas.

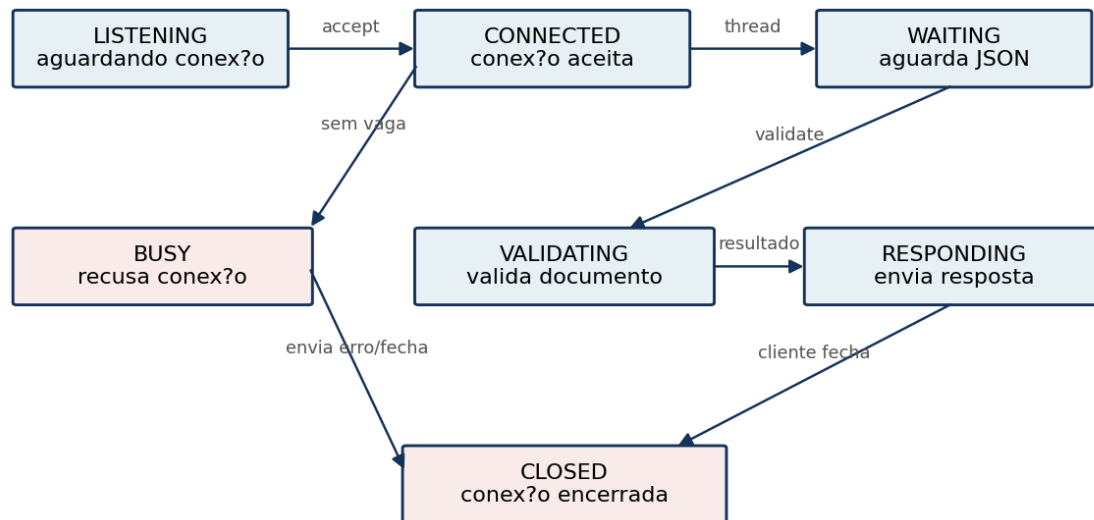
Cada documento possui dois dígitos verificadores. O servidor calcula uma soma ponderada, aplica módulo 11 e compara o resultado com o dígito recebido. Se o resto for menor que 2, o dígito esperado é 0; caso contrário, é 11 menos o resto.

7. Concorrência e proteção contra DoS

O servidor é multithread. Cada conexão aceita é atendida por uma thread daemon. Os sockets ativos são armazenados em um set dinâmico protegido, permitindo acompanhar e remover cada socket quando a conexão termina.

Um BoundedSemaphore controla a quantidade máxima de conexões simultâneas. O valor é configurável e possui padrão 32. Quando não há vaga, o servidor envia SERVER_BUSY e fecha o socket recusado.

8. Diagrama de estados finitos



9. Execução

```
python server.py --host 127.0.0.1 --port 5000 --max-connections 32
```

```
javac cliente.java
```

```
java cliente 127.0.0.1 5000 CPF 529.982.247-25
```

```
java cliente 127.0.0.1 5000 CNPJ 11.222.333/0001-81
```